

The Apache Software Foundation

Blogging in Action.

Apache Pig

SATURDAY FEB 23, 2013

Apache Pig: It goes to 0.11

After months of work, we are happy to announce the 0.11 release of Apache Pig. In this blog post, we highlight some of the major new features and performance improvements that were contributed to this release. A large chunk of the new features was created by Google Summer of Code (GSoC) students with supervision from the Apache Pig PMC, while the core Pig team focused on performance improvements, usability issues, and bug fixes. We encourage CS students to consider [applying for GSoC in 2013](#) -- it's a great way to contribute to open source software.

This blog post hits some of the highlights of the release. Pig users may also find a [presentation by Daniel Dai](#), which includes code and output samples for the new operators, helpful.

New Features

DateTime Data Type

The `DateTime` data type has been added to make it easier to work with timestamps. You can now do date and time arithmetic directly in a Pig script, use UDFs such as `CurrentTime`, `AddDuration`, `WeeksBetween`, etc. PigStorage expects timestamps to be represented in the [ISO 8601 format](#). Much of this work was done by Zhijie Shen as part of his GSoC project.

RANK Operator

The new [RANK operator](#) allows one to assign an ordinal number to every tuple in a relation. A user can specify whether she wants exact rank (elements with the same sort value get the same rank) or 'DENSE' rank (elements with the same sort value get consecutive rank values). One can also rank by a field value, in which case the relation is sorted by this field prior to ranks being assigned. Much of this work was done by Allan Avendaño as part of his GSoC project.

```
A = load 'data' AS (f1:chararray,f2:int,f3:chararray);
```

```
DUMP A;
(David,1,N)
(Tete,2,N)
(Ranjit,3,M)
```

Calendar

« January 2016

Sun	Mon	Tue	Wed	Thu	Fri	Sat
					1	2
3	4	5	6	7	8	9
10	11	12	13	14	15	16
17	18	19	20	21	22	23
24	25	26	27	28	29	30
31						

Today

Search

Hot Blogs (today's hits)

Tag Cloud

Categories

All
Music
General
Status

Feeds

All
/Music
/General
/Status
Comments

Links

[Apache Roller Project](#)
[Apache Software Foundation](#)

Navigation

[ASF Blogs](#)
[Weblog](#)
[Login](#)

```
(Ranjit,3,P)
(David,4,Q)
(David,4,Q)
(Jillian,8,Q)
(JaePak,7,Q)
(Michael,8,T)
(Jillian,8,Q)
(Jose,10,V)
```

```
B = rank A;
```

```
dump B;
(1,David,1,N)
(2,Tete,2,N)
(3,Ranjit,3,M)
(4,Ranjit,3,P)
(5,David,4,Q)
(6,David,4,Q)
(7,Jillian,8,Q)
(8,JaePak,7,Q)
(9,Michael,8,T)
(10,Jillian,8,Q)
(11,Jose,10,V)
```

```
C = rank A by f1 DESC, f2 ASC DENSE;
```

```
dump C;
(1,Tete,2,N)
(2,Ranjit,3,M)
(2,Ranjit,3,P)
(3,Michael,8,T)
(4,Jose,10,V)
(5,Jillian,8,Q)
(5,Jillian,8,Q)
(6,JaePak,7,Q)
(7,David,1,N)
(8,David,4,Q)
(8,David,4,Q)
```

CUBE and ROLLUP Operators

The new `CUBE` and `ROLLUP` operators of the equivalent SQL operators provide the ability to easily compute aggregates over multi-dimensional data. Here is an example:

```
events = LOAD '/logs/events' USING EventLoader() AS (lang, country, a
eventcube = CUBE events BY
  CUBE(lang, country), ROLLUP(app_id, event_id);
result = FOREACH eventcube GENERATE
  FLATTEN(group) as (lang, country, app_id, event_id),
  COUNT_STAR(cube), SUM(cube.total);
STORE result INTO 'cuberresult';
```

The `CUBE` operator produces all combinations of cubed dimensions. The `ROLLUP` operator produces all levels of a hierarchical group, meaning, `ROLLUP(country, region, city)` will produce aggregates by country, country and region, country, region, and city, but not country and city (without region). When used together as in the above example, the output groups will be the cross product of all groups generated by cube and rollup operation. That means that if there are m dimensions in cube operations and n dimensions in rollup operation then overall number of combinations will be $(2^m) * (n+1)$. Detailed documentation can be seen in [the CUBE Jira](#). This work was done by Prasanth Jayachandran as part of his GSoC project. He also did further work on optimizing the cubing computation to make it extremely scalable; this optimization will likely be added to the Pig 0.12 release.

Groovy UDFs

Pig has support for UDFs written in JRuby and Jython. In this release, support for [UDFs in Groovy](#) is added, providing an easy bridge for converting Groovy and Pig data types and specifying output schemas via annotations. This work was contributed by Mathias Herberts.

Improvements

Performance improvement of in-memory aggregation

Pig 0.10 introduced in-memory aggregation for algebraic operators -- instead of relying on Hadoop combiners, which involve writing map outputs to disk and post-processing them to apply the combine function, Pig can optionally buffer up map outputs in memory and apply combiners without paying the IO cost of writing intermediate data out to platters.

While the initial implementation significantly improved performance of a number of queries, we found some corner cases where it actually hurt performance; furthermore, reserving a large chunk of memory for aggregation buffers can have negative effects on memory-intensive tasks. In Pig 0.11, we completely [rewrote the partial aggregation operator](#) to be much more efficient, and [integrated it with Pig's Spillable Memory Manager](#), so it no longer requires dedicated space on the task heap. This feature is still considered experimental and is off by default; you can turn it on by setting `pig.exec.mapPartAgg` to `true`. With these changes in place, Twitter was able to turn this option on by default for all Pig scripts they run on their clusters -- thousands of Map-Reduce jobs per day (they also dropped `pig.exec.mapPartAgg.minReduction` to 3, to be even more aggressive with this feature).

Performance improvement related to Spillable management

Speaking of the `SpillableMemoryManager` -- it also saw some significant improvements. The default collection data structure in Pig is a "Bag". Bags are spillable, meaning that if there is not enough memory to hold all the tuples in a bag in RAM, Pig will spill part of the bag to disk. This allows a large job to make progress, albeit slowly, rather than crashing from "out of memory" errors. The way this worked before Pig 0.11 was as follows:

- Every time a Bag is created from the `BagFactory`, it is registered with the `SpillableMemoryManager`.
- The `SpillableMemoryManager` keeps a list of `WeakReferences` to `Spillable` objects
- Upon getting a notification that GC is about to happen, the `SMM` iterates through its list of `WeakReferences`, deleting ones that are no longer valid (pointing to null), and looking for the largest `Spillable` it can find. It then asks this `Spillable` to spill, and relies on the coming GC to free up spilled data.

Some users reported seeing large amounts of time taken up by traversing the `WeakReference` list kept by the `SMM`. A large `WeakReference` list affected both performance, since we had to iterate over large lists when GC was imminent, and memory, since each `WeakReference` adds 32 bytes of overhead on a 64-bit JVM. In Pig 0.11 we modified the Bag code so that instead of registering all bags in case they grow, we have Bags register themselves if their contents exceed 100KB, the logic being that a lot of bags will never reach this size, and would not be useful to spill anyway. This drastically reduced the amount of time and memory we spend on the `SpillableMemoryManager`.

Improvements to AvroStorage and HBaseStorage

HBase:

- Added the ability to set HBase scan `maxTimestamp`, `minTimestamp` and `timestamp` in `HBaseStorage`.
- Significant performance optimization for filters over many columns
- Compatibility with HBase 0.94 + secure cluster

Avro:

- `AvroStorage` can now optionally skip corrupt Avro files
- Added support for recursively defined records
- Added support for Avro 1.7.1
- Better support for file globbing

Faster, leaner Schema Tuples

Pig uses a generic `Tuple` container object to hold a “row” of data. Under the covers, it’s simply a `List`, where the objects might be `Strings`, `Longs`, other `Tuples`, `Bags`, etc. Such generality comes with overhead. We found that we can achieve significant performance gains in both size and speed of Tuples if, when the schema of a tuple is known, custom classes are auto-generated on the fly for working with particular schemas. You can see the [results of our benchmarks](#) (“Tuple” is the default generic tuple implementation; “Primitive” is an earlier attempt at tuple optimization, which streamlined handling of schemas consisting only of longs, ints, etc -- this approach was abandoned after the codegen work was complete; “Schema” is the codegen work). To turn on schema tuples, set the `pig.schematuple` property to `true`. This feature is considered experimental and is off by default.

New APIs for Algebraic, Accumulator UDFs, and Guava

We added a number of helper classes to make creating new UDFs easier, and improved some of the APIs around UDFs.

- Pig Tuple object is now a Java `Iterable`, so you can easily loop over all of the fields if you like (PIG-2724 has details)
- Accumulators can now terminate early. This can be a big performance win for accumulators that can bail out upon reaching some success condition. One simply has to implement the [TerminatingAccumulator interface](#), which has just one method: `isFinished()`.
- A new abstract class, [IteratingAccumulatorEvalFunc](#), has been added to make it easier to write Accumulator functions. To write an Accumulator, one can simply extend this abstract class and implement a single method which takes an `Iterator` and returns the desired result. If you’ve implemented Accumulators before, you’ll see why the sample code in [PIG-2651](#) is much cleaner and simpler to write.
- Instead of implementing a `getOutputSchema` function, UDF authors can tell Pig their output schema by annotating the UDF with an [@OutputSchema](#) annotation
- Before Pig 0.11 if you wanted to implement an Algebraic or Accumulative UDF, you still had to implement the regular `exec()` method, as well. We’ve introduced a couple of new abstract classes, [AlgebraicEvalFunc](#) and [AccumulatorEvalFunc](#), which give you the derivable implementations for free. So if you implement `AlgebraicEvalFunc`, you automatically get the regular and Accumulator implementations. Saves code, saves sanity!
- We’ve found that many Pig users are interested in being able to share their UDF logic with non-Pig programs.

[FunctionWrapperEvalFunc](#) allows one to easily wrap Guava functions which contain the core logic, and keep UDF-specific code minimal.

- We've found that many UDFs work on just a single field (as opposed to a multi-field tuple), and return a single value. For those cases, extending [PrimitiveEvalFunc<IN, OUT>](#) allows the UDF author to skip all the tuple unwrapping business and simply implement `public OUT exec(IN input)`, where IN and OUT are primitives.
- We find ourselves wrapping `StoreFuncs` often and have created [StoreFuncWrapper](#) and [StoreFuncMetadataWrapper](#) classes to make this easier. These classes allow one to subclass and decorate only select `StoreFunc` methods.
- `mock.Storage`, a helper `StoreFunc` to simplify writing JUnit tests for your pig scripts, was quietly added in 0.10.1 and got a couple of bug fixes in 0.11. See details in [mock.Storage docs](#).

Other Changes

A number of other changes were introduced -- optimizations, interface improvements, small features, etc. Here is a sampling:

- Penny, a debugging tool introduced as an experimental feature in Pig 0.9, has been removed due to lack of adoption and complexity of the codebase
- `StoreFuncs` can now implement a new method, `cleanupOnSuccess`, in addition to the previously existing `cleanupOnFailure`
- Pig Streaming can be passed values from the `JobConf` via environment variables. Rather than pass all the variables from the `JobConf`, which can cause Java's `ProcessBuilder` to croak for large enough `JobConf`s, Pig 11 allows the user to explicitly specify which properties should be passed in by setting the value of `pig.streaming.environment` property
- The logic used to estimate how many reducers should be used for a given Map-Reduce job is now pluggable, with the default implementation remaining as it was.
- Setting the `pig.udf.profile` property to true will turn on counters that approximately measure the number of invocations and milliseconds spent in all UDFs and Loaders. Use this with caution, as this feature can really bloat the number of counters your job uses! Useful for lightweight debugging of jobs.
- Local mode is now significantly faster
- Merge Join previously only worked immediately after loading. It is now allowed after an `ORDER` operator
- Grunt prints schemas in human-friendly JSON if you set `pig.pretty.print.schema=true`
- Better HCatalog integration
- Pluggable `PigProgressNotificationListeners` allow custom tool integration for monitoring Pig job progress -- for an example of what can be possible with this, check out [Twitter Ambrose](#)
- Extensive work went into making sure that Pig works with JDK 7 and Hadoop 2.0

A lot of work went into this release, and we are grateful to all the contributors.

We hope you like all the new stuff! Let us know what you think -- users@pig.apache.org.

Dmitriy Ryaboy ([@squarecog](#)) on behalf of the Pig team.

Posted at 02:31AM Feb 23, 2013 by dvryaboy in General | [Comments\[0\]](#) |  

Comments:

Post a Comment:

Name:

E-Mail:

URL:

- ☐ Notify me by email of new comments
- ☐ Remember Information?

Your Comment:

HTML Syntax: **NOT allowed**

Please answer this simple math question

6 + 84 =

Preview

Post